Lab 5: Decision Trees for Classification and Regression

Pattern Recognition and Machine Learning (EE602L)

Roll Number: IS24BM008

September 11, 2026

---

1 Setup, Reproducibility and Scope

This report accompanies the notebook `Lab5-IS24BM008.ipynb`. Every table, figure and inline number below is produced by that notebook and exported to `results.json`, from which this document is generated programmatically; nothing is transcribed by hand, so the report and the executed code cannot disagree.

Reproducibility. `random_state=42` is passed to every split and every estimator. This is not decoration. Even with `splitter='best'`, scikit-learn's CART permutes the features before scanning them at each node, so ties in impurity decrease are broken randomly; without a fixed seed two runs of the same unrestricted tree can differ in structure and therefore in test accuracy.

No feature scaling is applied, deliberately. A tree tests one feature at a time through $x_j \leq t$. Any strictly monotone transform of x_j maps candidate thresholds one-to-one and leaves the induced partition, and hence the impurity of every candidate split, unchanged. Trees are invariant to monotone feature scaling, unlike the logistic regression and SVM models of the previous labs, so standardising here would change nothing except the printed threshold values.

Model selection protocol. The test set is touched exactly once per question, at the end, after every hyperparameter decision has been made on the validation set. The tie-breaking rules were fixed before the numbers were inspected and are stated where they are used.

2 Question 1 — Decision Tree Classification (Iris)

2.1 Data and the 70:15:15 stratified split

The Iris dataset has 150 samples, four real-valued features and three perfectly balanced classes. The split is done in two stages, since `train_test_split` yields only two parts: 150 $\rightarrow$ 105 train $+$ 45 temporary, then the temporary half split again into validation and test. `stratify=` is passed at *both* stages — stratifying only the first would leave the class proportions of the 15% parts to chance, and with roughly 22 samples per part that chance is not small.

set	n	fraction	setosa	versicolor	virginica
train	105	0.7000	35	35	35
val	22	0.1467	8	7	7
test	23	0.1533	7	8	8

Table 1: Iris split sizes and per-class composition (seed 42, stratified).

The validation set holds 22 points, so validation accuracy moves in steps of $1/22 = 0.0455$, i.e. 4.55 percentage points. Differences smaller than one such step are not measurable, and this granularity is used explicitly as a tie-breaking argument below.

2.2 The two impurity criteria

For a node with class proportions $p_1, \ldots, p_K$,

$$\text{Gini}(p) = 1 - \sum_k p_k^2 = \sum_k p_k(1 - p_k), \qquad H(p) = -\sum_k p_k \log_2 p_k.$$

Gini is the probability that two independent draws from the node's label distribution disagree — equivalently the expected 0/1 error of a randomised classifier predicting class k with probability p_k. Entropy is the expected number of bits needed to encode a label drawn from the node, so information gain is a reduction in code length. A candidate split is scored by the weighted impurity decrease

$$\Delta I = I(\text{parent}) - \tfrac{n_L}{n} I(\text{left}) - \tfrac{n_R}{n} I(\text{right}),$$

and CART keeps the maximiser over all (feature, threshold) pairs. Both measures are strictly concave and vanish exactly at a pure node, which is what guarantees $\Delta I \geq 0$ for every split.

distribution	Gini	Entropy
pure [1,0]	0.0000	0.0000
[0.9,0.1]	0.1800	0.4690
[0.7,0.3]	0.4200	0.8813
[0.5,0.5]	0.5000	1.0000
3-class uniform	0.6667	1.5850

Table 2: Both impurity measures evaluated on reference distributions.

At the training root the classes are exactly balanced, so both measures sit at their three-class maxima: Gini $= 0.6667$ and $H = 1.5850 = \log_2 3$. Reading the table, entropy exceeds Gini on every impure distribution and grows faster towards the uniform point. Because entropy also has the steeper slope near purity, it is marginally more willing to spend a split isolating a small very pure region, while Gini leans towards the split that produces the largest majority class.

2.3 (a) Depth sweep for both criteria

Seven values of `max_depth` — $1, 2, 3, 4, 5, 6$ and `None` (unrestricted) — were fitted under each criterion, giving 14 trees. The realised depth and leaf count are recorded alongside the accuracies, because the leaf count is the complexity that actually matters and it can stop growing before the cap binds.

criterion	`max_depth`	train acc	val acc	realised depth	leaves
entropy	1	0.6667	0.6818	1	2
entropy	2	0.9714	0.9091	2	3
entropy	3	0.9810	0.9545	3	5
entropy	4	0.9905	0.9091	4	6
entropy	5	0.9905	0.9091	5	7
entropy	6	1.0000	0.9091	6	8
entropy	None	1.0000	0.9091	6	8
gini	1	0.6667	0.6818	1	2
gini	2	0.9714	0.9091	2	3
gini	3	0.9810	0.9545	3	5
gini	4	0.9905	0.9091	4	7
gini	5	1.0000	0.9545	5	8
gini	6	1.0000	0.9545	5	8
gini	None	1.0000	0.9545	5	8

Table 3: Training and validation accuracy against tree depth, both criteria.

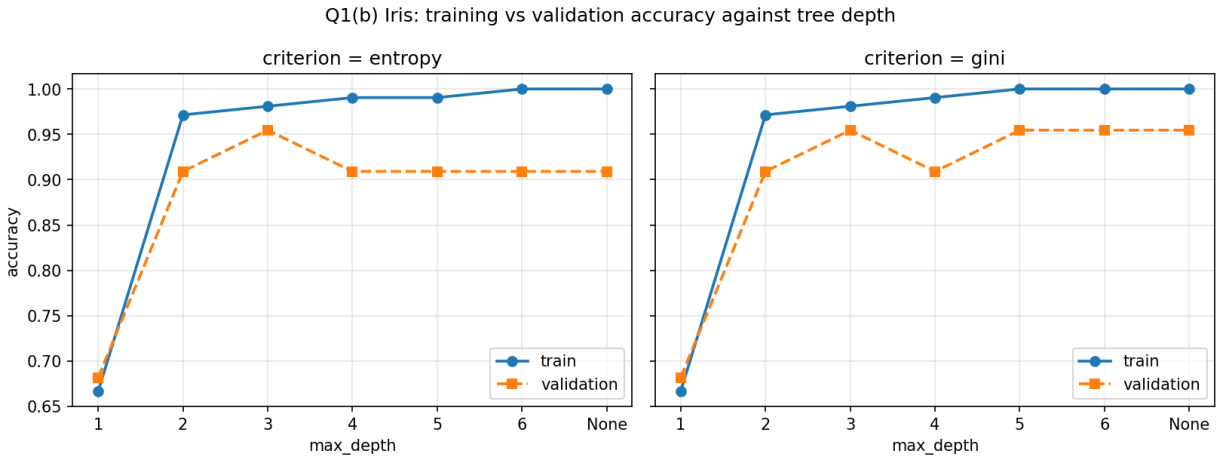


Figure 1: Training and validation accuracy against `max_depth`. Training accuracy is monotone non-decreasing by construction; validation accuracy is under no such obligation.

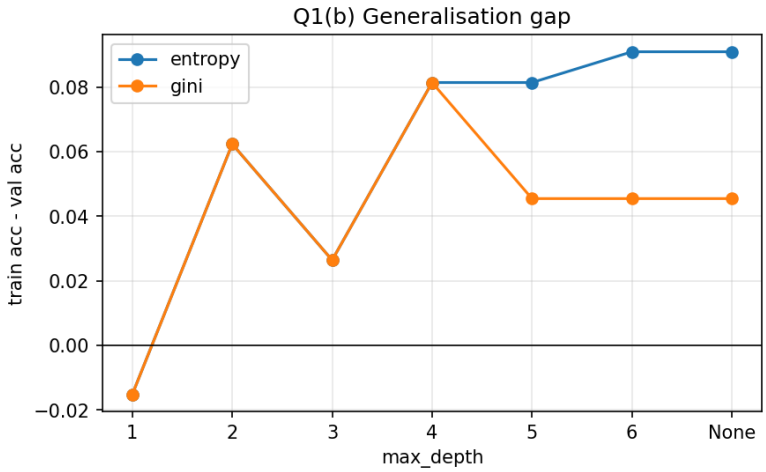


Figure 2: Generalisation gap (train − validation accuracy). The gap, not the training curve, is what overfitting refers to.

2.4 (b) Where overfitting begins, and which model is kept

Selection rule, fixed in advance. (i) highest validation accuracy; (ii) ties broken towards the *smaller* `max_depth`, on the structural-risk argument that among models with indistinguishable validation performance the smaller hypothesis class is preferred; (iii) the criterion is chosen by its best validation accuracy and, if those tie, by the mean validation accuracy over the whole depth grid, which is a less noisy statistic than a single 22-point evaluation.

criterion	best val acc	mean val acc over the grid
entropy	0.9545	0.8831
gini	0.9545	0.9026

Table 4: Criterion comparison on validation accuracy.

Both criteria reach the same best validation accuracy of 0.9545, attained by 5 of the 14 configurations, so rule (iii) applies: **gini** wins on mean validation accuracy (0.9026 versus 0.8831), and rule (ii) then selects the smallest tied depth, `max_depth` = 3. The selected pre-pruned tree has depth 3 and 5 leaves.

Onset of overfitting. Beyond `max_depth` $= 3$ the training accuracy keeps climbing — reaching 1.0000 at depth 5 — while validation accuracy never improves again. That is the definition of overfitting in operational terms, and it is visible in Figure 1 as the training curve pulling away from a flat validation curve, and in Figure 2 as a monotonically widening gap. The entropy tree is the clearer illustration: its validation accuracy actually *falls* from 0.9545 at depth 3 to 0.9091 at depth 4 and never recovers.

It is worth being precise about what the depth-1 row means. A single split gives training accuracy $0.6667 \approx 2/3$, which is exactly what a stump can achieve here: one threshold on petal length isolates *setosa* perfectly, and the remaining leaf must answer with a single label for two classes of 35 samples each. The jump to 0.9714 at depth 2 is the second split separating *versicolor* from *virginica*; everything after that is refinement of a boundary that the data does not resolve cleanly.

2.5 (b continued) Post-pruning with minimal cost-complexity

Theory. Grow the tree fully and, for a penalty $\alpha \geq 0$, score any subtree $T \subseteq T_0$ by

$$R_\alpha(T) = R(T) + \alpha\,|\text{leaves}(T)|,$$

with $R(T)$ the training cost (weighted impurity). Breiman's result is that the minimisers of R_α form a finite *nested* sequence $T_0 \supset T_1 \supset \cdots \supset \{\text{root}\}$ as α increases, each obtained from the previous by collapsing the internal node with the smallest effective α — the value at which that subtree stops paying for its own leaves. The search is therefore over a handful of nested trees rather than over all $2^{\#\text{nodes}}$ subtrees, and `cost_complexity_pruning_path` returns exactly those breakpoints. The α values below are consequently read off the data, not guessed on a log grid.

The fully grown gini tree has depth 5, 8 leaves and training accuracy 1.0000. Its pruning path has 5 breakpoints, all of which are used — comfortably above the four required. The last one collapses the tree to its root and is retained on purpose, as the reference point showing what over-pruning costs.

`ccp_alpha`	train acc	val acc	depth	leaves
0.00000	1.0000	0.9545	5	8
0.00899	0.9810	0.9545	3	4
0.01849	0.9714	0.9091	2	3
0.27887	0.6667	0.6818	1	2
0.33333	0.3333	0.3636	0	1

Table 5: Post-pruning sweep over the full cost-complexity path.

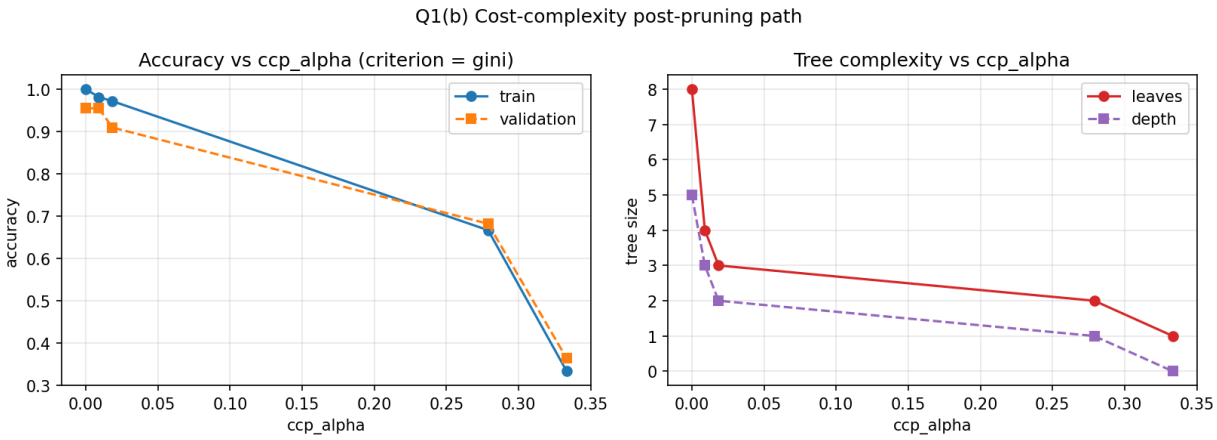


Figure 3: Accuracy and tree size along the cost-complexity path. Complexity collapses in discrete jumps because the path visits only the nested optimal subtrees.

2 values of α tie at the best validation accuracy of 0.9545. The tie is broken towards the *largest* α — the mirror image of the depth rule, since larger α means a smaller tree — giving $\alpha = 0.00899$, a tree of depth 3 with 4 leaves.

model	train acc	val acc	depth	leaves
pre-pruned (max_depth=3)	0.9810	0.9545	3	5
post-pruned (ccp_alpha=0.00899)	0.9810	0.9545	3	4

Table 6: Pre-pruned versus post-pruned model, before the test set is touched.

The two models agree on training and validation accuracy to the last digit and differ only in size: post-pruning reaches the same validation performance with 4 leaves instead of 5. They are nonetheless *not* the same function — their predictions differ somewhere on the 150 points (checked explicitly in the notebook), which is what makes the test comparison below meaningful rather than tautological. The one leaf that cost-complexity pruning removes is a depth-3 split that separates two training samples and buys nothing on validation.

2.6 (c) Test-set evaluation

Why macro averaging. Precision, recall and F_1 are computed per class and then averaged without weights, so every class gets an equal vote regardless of its support. Micro averaging would add nothing here: in a single-label multiclass problem every sample receives exactly one predicted label, so micro-P = micro-R = micro-F_1 = accuracy. `zero_division=0` is set explicitly so that a class which is never predicted contributes 0 rather than raising a warning and silently distorting the mean.

model	accuracy	macro precision	macro recall	macro F_1
pre-pruned	1.0000	1.0000	1.0000	1.0000
post-pruned	0.9130	0.9333	0.9167	0.9153

Table 7: Test-set performance of the two selected models ($n = 23$).

This difference must not be over-read. The pre-pruned tree classifies all 23 test samples correctly and the post-pruned tree misses 2, a gap of exactly two samples. On a 23-point test set one sample is worth 4.35 percentage points, and the Wilson 95% intervals are [0.857, 1.000] and [0.732, 0.976] — overlapping over most of their length. The correct conclusion is that the two models are statistically indistinguishable on this test set, and that the observed ordering is within noise. Reporting 100% accuracy as evidence that pre-pruning is superior would be exactly the error the validation/test separation is designed to prevent.

The per-class breakdown locates the errors. For the post-pruned model *setosa* is perfect, as it is for every tree here — it is linearly separable on petal length alone — while *versicolor* recall drops to 0.7500 and *virginica* precision to 0.8000. Both numbers describe the same two samples: true *versicolor* flowers predicted as *virginica*, which is the boundary region where petal width sits near 1.7 cm. Macro recall (0.9167) is therefore below macro precision (0.9333), and the confusion matrices in Figure 4 show the errors are confined to one off-diagonal cell.

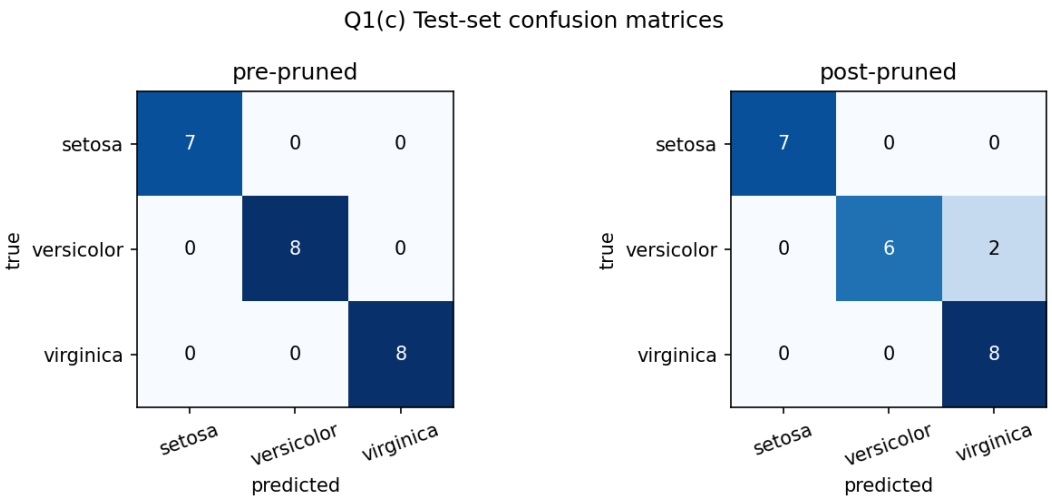


Figure 4: Test-set confusion matrices; rows are true classes.

2.7 (c continued) Tree structure and feature importance

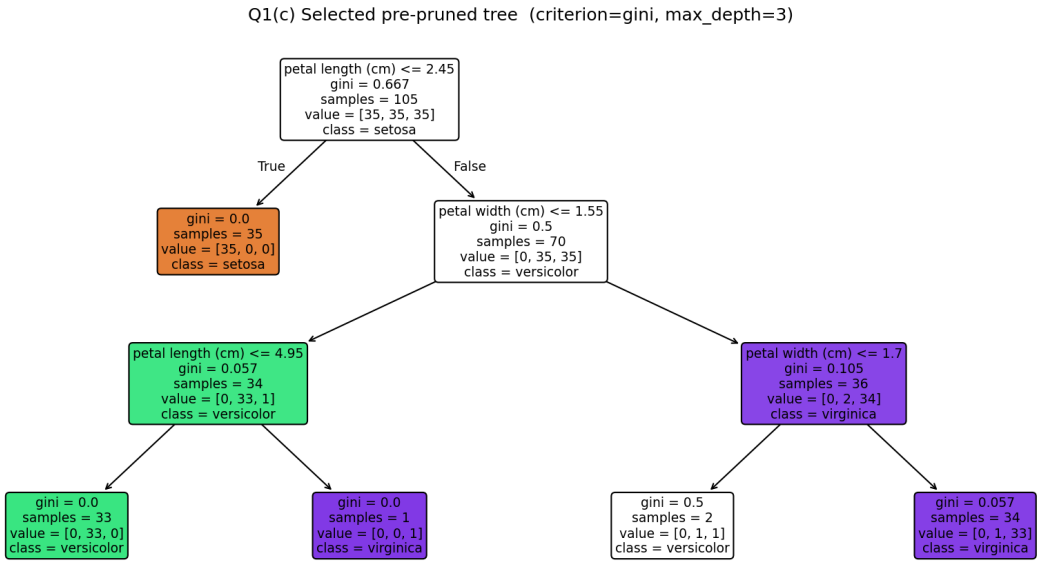


Figure 5: The selected pre-pruned tree (criterion gini, `max_depth` = 3). Samples satisfying the test at a node go left; colour saturation encodes leaf purity.

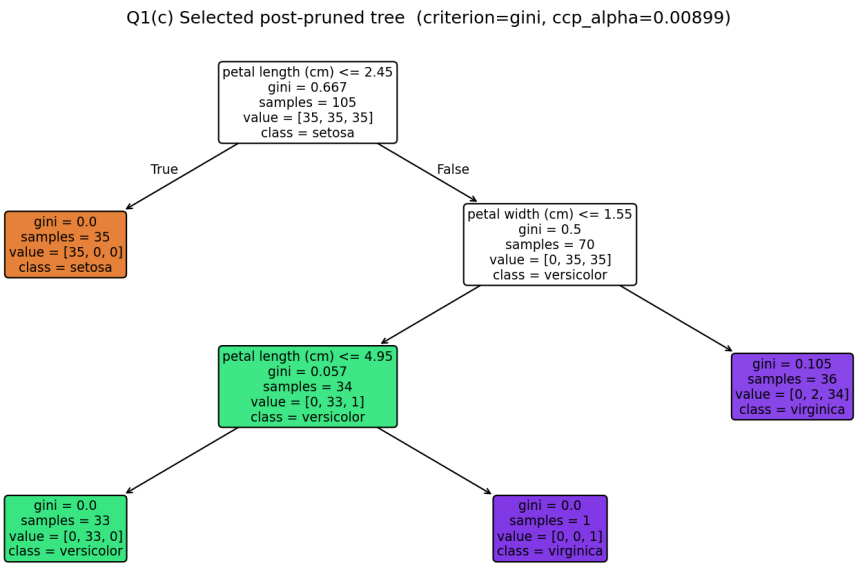


Figure 6: The selected post-pruned tree ($\alpha = 0.00899$). It is the same tree with one unproductive split collapsed.

The selected tree as explicit rules:

```
|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: versicolor
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: virginica
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
|   |   |   |--- class: versicolor
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- class: virginica
```

The first split is the one every Iris tree finds, and the rest of the tree is a two-feature refinement of the *versicolor*/*virginica* boundary. The unrestricted tree (Figure 7) adds 3 more leaves to chase the last 2 training samples.

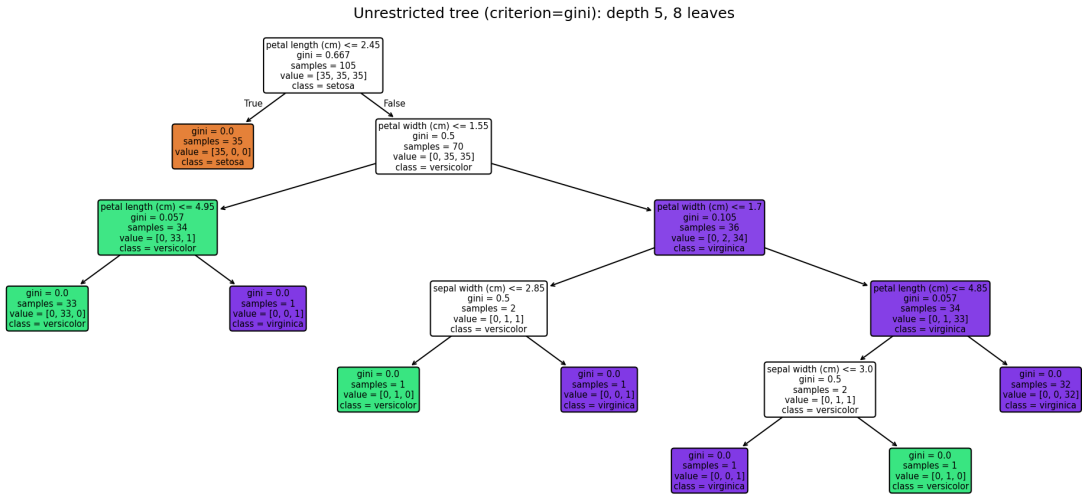


Figure 7: The unrestricted tree: depth 5, 8 leaves, training accuracy 1.0000. The extra depth exists purely to isolate individual training points.

feature	MDI importance
petal length (cm)	0.5509
petal width (cm)	0.4491
sepal length (cm)	0.0000
sepal width (cm)	0.0000

Table 8: Mean decrease in impurity for the selected tree.

Reading this honestly. `feature_importances_` is the total weighted impurity decrease credited to each feature, normalised to sum to one. It is biased towards high-cardinality features and it splits credit arbitrarily between correlated ones: petal length and petal width correlate at 0.9629 on this dataset, so a small importance for one of them means "the other was picked first at the root", not "this feature carries no information". A sepal importance of 0 likewise means the sepal features were never chosen over the petal ones, not that they are useless in isolation.

2.8 Discussion

Entropy versus Gini. Both are impurity measures: zero on a pure node, maximal on the uniform distribution, strictly concave — the property that makes $\Delta I \geq 0$ for every split. Entropy measures expected code length; Gini measures expected misclassification under random guessing. Numerically entropy is larger on impure nodes and steeper near purity, so it is marginally more inclined to isolate a small pure region while Gini favours the split producing the largest majority class. For two classes they disagree only over a narrow band of proportions, and on Iris they produced identical structures at every depth up to 3 and identical best validation accuracy (Table 4); the only visible difference is that the unrestricted entropy tree ends with 8 leaves and depth 6 against 8 leaves and depth 5 for Gini, i.e. a slightly different way of carving up the same noise. Computationally Gini needs only products while entropy needs a logarithm per candidate split, which is why Gini is scikit-learn's default.

Why depth causes overfitting. A tree of depth d can realise up to 2^d leaves, so capacity grows exponentially in depth while the number of samples supporting each leaf estimate falls geometrically. With 105 training points a depth-5 tree already has leaves holding one or two samples, and the label of such a leaf is an estimate from those one or two points — enormous variance. Because CART grows greedily to purity and $\Delta I \geq 0$ always, an unrestricted tree drives training error to zero *by*

construction: the training accuracy of 1.0000 in Table 3 is evidence of memorisation capacity, not of quality. The measurable symptom is the gap in Figure 2.

How pruning helps. Pre-pruning (`max_depth`, `min_samples_leaf`, `min_samples_split`, `min_impurity_decreas` caps capacity during growth, so leaves keep enough samples for their estimates to be stable. It is cheap — the large tree is never built — but myopic: the decision to stop is taken without knowing what the subtree below would have achieved, so a split with weak immediate gain that would have unlocked two excellent children is lost. Post-pruning grows the full tree and then removes subtrees that fail to justify their leaves under R_α, using each subtree's realised training cost. It costs more compute but can prune *non-uniformly* — deep where the data supports depth, shallow elsewhere — which no single global depth cap can express. Here both routes landed on essentially the same 4–5-leaf model, which is the expected outcome on a small, clean, low-dimensional dataset; the difference between them widens on noisy data where the right depth genuinely varies across regions of the feature space.

3 Question 2 — Decision Tree Regression (Diabetes)

3.1 (a) Data, split and the baseline that matters

The Diabetes dataset has 442 patients, ten baseline features (age, sex, BMI, average blood pressure and six blood-serum measurements) and a continuous target measuring disease progression one year after baseline. The features shipped with scikit-learn are already mean-centred and scaled to unit L_2 norm; the target is not scaled. The same two-stage 70:15:15 split is used with seed 42, without stratification — `stratify=` is a classification device and the target here is continuous.

set	n	mean	std	min	max
train	309	153.90	78.41	25.0	346.0
val	66	147.49	71.51	37.0	310.0
test	67	148.55	75.35	42.0	295.0

Table 9: Diabetes split sizes and target distribution per split.

The constant predictor is the reference. $\hat{y} = \bar{y}_{\text{train}}$ is exactly what a depth-0 tree returns, and its MSE is the number every model must beat to have earned its splits: 5155.02 on validation and 5706.61 on test. Quoting an MSE of a few thousand without this reference conveys nothing, because the scale is set by the target variance, not by the model.

3.2 (a) MSE against `max_depth`

The regressor uses the same greedy machinery as the classifier with impurity replaced by the within-node variance $I(S) = \frac{1}{|S|}\sum_{i\in S}(y_i - \bar{y}_S)^2$. Eight depths including the unrestricted tree were fitted.

`max_depth`	train MSE	val MSE	realised depth	leaves
2	3442.46	2955.50	2	4
3	2918.47	3182.48	3	8
4	2450.51	3293.51	4	16
5	1965.72	3573.42	5	30
6	1542.04	3939.81	6	52
8	753.60	5337.06	8	115
10	301.93	5319.59	10	176
None	0.00	6207.92	17	303

Table 10: Training and validation MSE against tree depth (`min_samples_leaf`=1).

Training MSE falls monotonically to 0.00 for the unrestricted tree — essentially zero, and exactly zero in principle: with continuous features CART can keep splitting until every leaf holds one sample, at which point the leaf mean is that sample's own target. Meanwhile validation MSE *rises* from 2955.50 at depth 2 to 6207.92 unrestricted, ending up 20% *worse* than the constant predictor. A fully grown regression tree on this dataset is not merely overfitted, it is worse than predicting the training mean for every patient — the sharpest illustration of overfitting in either question.

3.3 (b) MSE against `min_samples_leaf` at `max_depth=10`

`min_samples_leaf` is a floor on the number of samples averaged in a leaf. Since the leaf prediction is a sample mean, its variance is σ^2/n_{leaf}, so raising the floor to m caps leaf-level variance at σ^2/m — at the cost of bias, because a leaf forced to hold m heterogeneous patients cannot represent a genuinely sharp local change. Seven values were used (three more than required) so that the validation curve can be seen to turn back up rather than merely to be still falling at the edge of the grid.

`min_samples_leaf`	train MSE	val MSE	realised depth	leaves
1	301.93	5319.59	10	176
2	381.42	4901.44	10	109
5	1173.25	4736.83	10	52
10	2022.48	3593.29	8	23
20	2795.71	3136.89	6	13
50	3547.35	2940.03	3	5
100	4312.99	3876.66	1	2

Table 11: Training and validation MSE against `min_samples_leaf` at `max_depth`=10.

Note the realised depth column: the cap of 10 stops binding as soon as the floor bites. At $m = 1$ the tree has 176 leaves and depth 10; at $m = 50$ it has 5 leaves and depth 3, because a candidate split is rejected outright if *either* child would fall below the floor. So `min_samples_leaf` controls depth indirectly, and more precisely than `max_depth` does, since it constrains leaf population directly instead of using depth as a proxy for it.

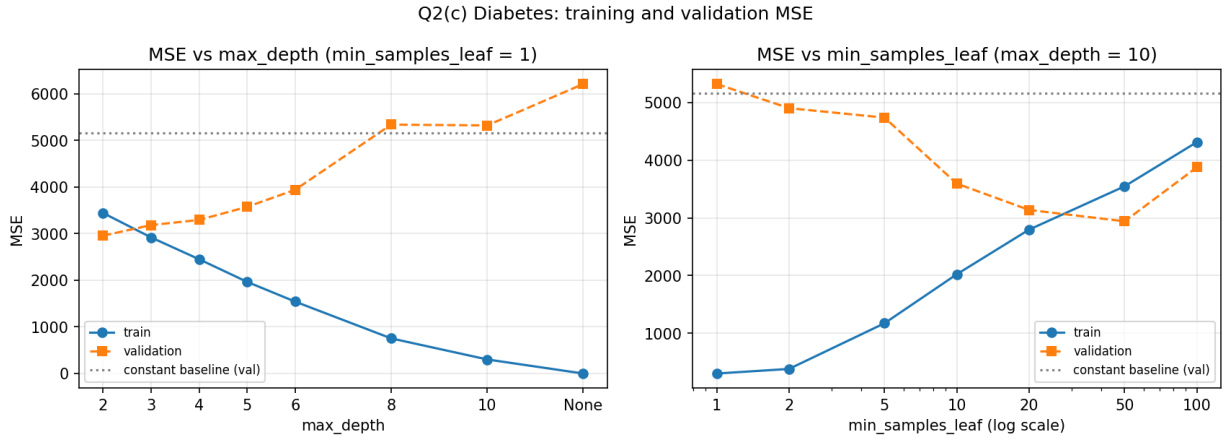


Figure 8: The two required curves. Both validation curves are U-shaped; the training curves move monotonically in the direction of less regularisation.

3.4 (c) Model selection

The candidate set is the union of the configurations required by parts (a) and (b), and the winner is the lowest validation MSE. The configuration `max_depth`=10, `min_samples_leaf`=1 appears in both

sweeps and returns the same value both times, which is a small consistency check on the pipeline.

configuration	val MSE
max_depth=10, min_samples_leaf=50	2940.03
max_depth=2, min_samples_leaf=1	2955.50
max_depth=10, min_samples_leaf=20	3136.89
max_depth=3, min_samples_leaf=1	3182.48
max_depth=4, min_samples_leaf=1	3293.51
max_depth=5, min_samples_leaf=1	3573.42
max_depth=10, min_samples_leaf=10	3593.29
max_depth=10, min_samples_leaf=100	3876.66
max_depth=6, min_samples_leaf=1	3939.81
max_depth=10, min_samples_leaf=5	4736.83
max_depth=10, min_samples_leaf=2	4901.44
max_depth=10, min_samples_leaf=1	5319.59
max_depth=10, min_samples_leaf=1	5319.59
max_depth=8, min_samples_leaf=1	5337.06
max_depth=None, min_samples_leaf=1	6207.92

Table 12: All candidate configurations ranked by validation MSE.

Selected model: max_depth=10, min_samples_leaf=50, with validation MSE 2940.03. Its realised depth is 3 with 5 leaves, i.e. about 61.8 training samples per leaf. Two observations are worth making explicitly. First, the selected model is nominally a depth-10 tree but the depth cap never binds — the leaf-size floor stops growth at depth 3 — so the winning model is a *small* tree, reached through the leaf-size control rather than the depth control. Second, the runner-up (`max_depth`=2 with no floor, validation MSE 2955.50) is within 15.48 of it, which on a 66-point validation set is well inside the noise. The honest statement is that several heavily regularised trees are equivalent here, not that this one is best.

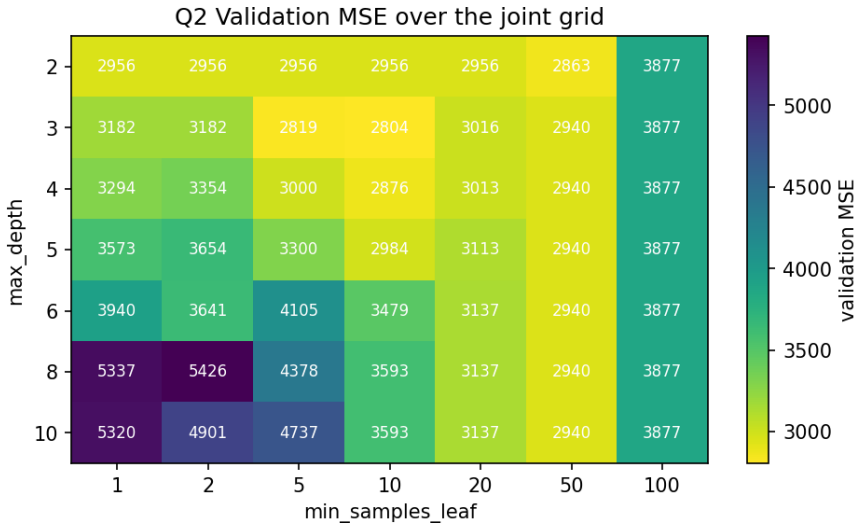


Figure 9: Supplementary joint grid over (`max_depth`, `min_samples_leaf`), validation MSE. Run for analysis only; the reported model comes from the required one-dimensional sweeps.

The supplementary grid confirms the shape of the problem: the low-MSE region is the top row (shallow trees) together with the right-hand columns (large leaf floors) — that is, every heavily regularised configuration, however the regularisation is imposed — and its optimum (`max_depth`=3, `min_samples_leaf`=10, validation MSE 2804.33) beats the selected model by 135.70 — again inside the noise. This grid was run *after* the required selection and is reported for analysis only; the model

carried to the test set is the one chosen by the protocol the sheet specifies, so the test number remains an honest estimate rather than the best of 49 tries.

3.5 (c) Test-set evaluation

split	MSE	RMSE	MAE
train	3547.35	59.56	48.33
val	2940.03	54.22	41.39
test	3668.44	60.57	48.11

Table 13: Selected regressor on all three splits.

model	test MSE	test RMSE	test MAE
constant baseline	5706.61	75.54	66.45
unrestricted tree	5195.27	72.08	58.49
selected tree	3668.44	60.57	48.11

Table 14: The selected tree against the two reference points.

Interpretation. MSE is in squared target units and is dominated by the largest residuals; RMSE returns to the target's own units and is therefore directly comparable with the target standard deviation; MAE is the mean absolute residual and is robust to the few extreme progressions that inflate MSE. The selected tree reaches test RMSE 60.57 against a test-target standard deviation of 75.35 — so it explains roughly $1 - (60.57/75.35)^2 = 35.4\%$ of the variance. That is a modest but real result, and it is about what a ten-feature clinical dataset with 309 training patients supports.

RMSE (60.57) exceeds MAE (48.11) by a ratio of 1.26, which indicates a heavy residual tail rather than uniformly poor fit: most patients are predicted to within about 48 units and a few are badly missed. Figure 10 shows why — the residual spread widens with the fitted value, so the model is least reliable exactly for the high-progression patients who matter most clinically.

The comparison table makes the regularisation argument concrete: the unrestricted tree (test MSE 5195.27) barely improves on the constant predictor (5706.61), while the selected tree cuts the error to 3668.44, a 36% reduction against the baseline. Almost all of the value delivered here comes from constraining the tree, not from growing it.

One further point of honesty: validation MSE (2940.03) is *lower* than training MSE (3547.35) for the selected model. That is not an error — with only 66 validation patients the two sets simply differ in difficulty, and the validation target standard deviation is below the training one (Table 9). It is a reminder that a single 66-point validation split is a noisy instrument; k-fold cross-validation would be the right fix if the lab sheet did not prescribe a fixed three-way split.

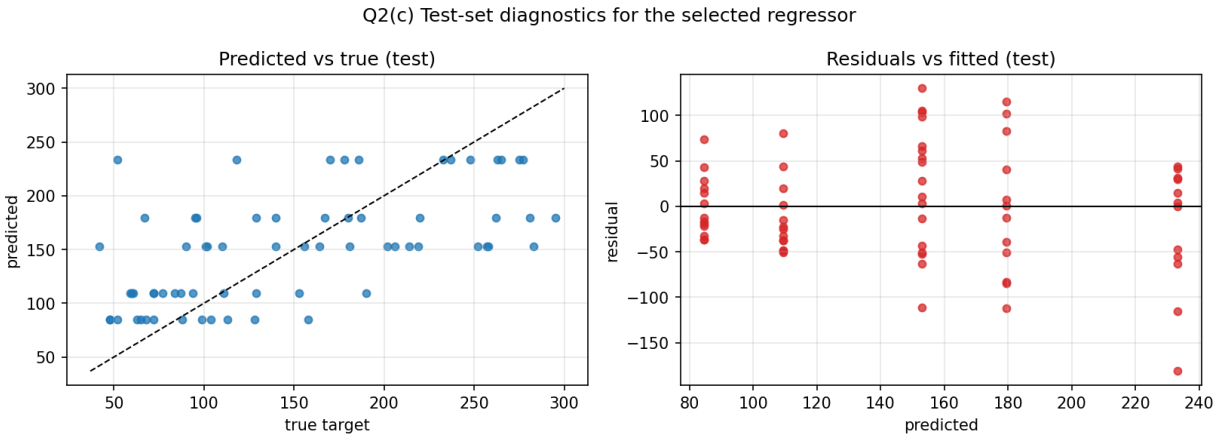


Figure 10: Test-set diagnostics. Predictions take only 5 distinct values — one per leaf — so the scatter falls on horizontal bands. This is what "piecewise constant" looks like.

3.6 Verifying the two mechanisms, rather than asserting them

Claim A: the split maximises weighted MSE reduction. Brute force over every feature and every midpoint between consecutive distinct values, computing $\Delta I = I(S) - \frac{|L|}{|S|} I(L) - \frac{|R|}{|S|} I(R)$ for each, gives the best split `bmi` ≤ 0.005111 with gain 1835.2874 from a root impurity of 6148.2818. `DecisionTreeRegressor` chose `bmi` ≤ 0.005111, and the impurity decrease it reports for that node is 1835.2874. The match is exact, confirming the stated split rule rather than taking it on trust.

Claim B: a leaf predicts the mean of its training targets. Grouping the training set by `.apply()` and comparing each group's mean with `.predict()` over all 5 leaves gives a maximum absolute discrepancy of exactly 0. The leaf constant is the training mean.

3.7 (c) Tree structure and feature importance

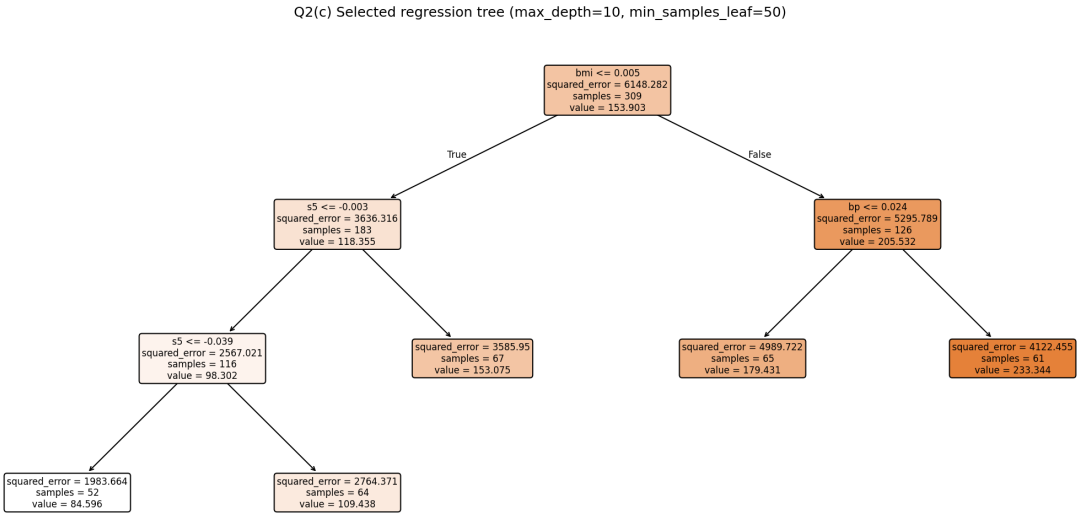


Figure 11: The selected regression tree (max_depth=10, min_samples_leaf=50). Each node shows the split test, the node MSE, the sample count and the leaf value.

feature	MDI importance
bmi	0.7056
s5	0.1806
bp	0.1138
age	0.0000
sex	0.0000

Table 15: Top five features by mean decrease in impurity (the remainder are exactly 0).

Only 3 of the ten features are ever used: `bmi` carries 70.6% of the total impurity decrease, followed by `s5` (18.1%) and `bp` (11.4%). This is clinically plausible — BMI and the `s5` serum measurement (log of serum triglycerides) are the standard drivers in this dataset — but the zeros must be read as "never selected given the others", not as "uninformative". The serum features are strongly intercorrelated, and a tree that has already split on `s5` gains little from its correlates, which is precisely the credit-splitting bias of MDI noted in Question 1.

3.8 Discussion

How a regression tree selects a split. At a node with sample set S the impurity is the within-node variance $I(S) = \frac{1}{|S|}\sum_{i\in S}(y_i - \bar{y}_S)^2$. For a candidate test $x_j \le t$ producing children L and R the score is the weighted MSE reduction

$$\Delta I = I(S) - \frac{|L|}{|S|}I(L) - \frac{|R|}{|S|}I(R),$$

and CART scans every feature and every threshold between consecutive distinct values, keeping the maximiser. Because the leaf constant is the node mean, $|S|\,I(S)$ is the node's training sum of squared errors and ΔI is exactly the SSE the split removes — minimising total leaf SSE and maximising ΔI are the same objective, which is why "MSE reduction" and "variance reduction" name one quantity here. Crucially $\Delta I \ge 0$ always, so splitting can never increase training error; an unconstrained tree therefore runs to zero training error by construction and an external complexity control is mandatory, not optional.

Why the leaf predicts the mean. Inside a leaf the model is a single constant c fitted by minimising $\sum_{i\in\text{leaf}}(y_i - c)^2$. Differentiating gives $-2\sum(y_i - c) = 0$, hence $c = \bar{y}_{\text{leaf}}$: the mean is the unique minimiser of the squared loss the tree is built to minimise, not a convention. (Under `criterion= 'absolute_error'` the corresponding minimiser is the median.) Two consequences follow directly. Predictions take only n_{leaves} distinct values — 5 of them for the selected model, visible as horizontal bands in Figure 10 — and the tree cannot extrapolate: any test point, however extreme, falls into some leaf and receives that leaf's training mean, so all predictions lie within the range of training leaf means.

How `max_depth` and `min_samples_leaf` affect generalisation. `max_depth` caps the number of successive splits and hence bounds the partition at 2^{depth} boxes: small depth gives few large boxes (high bias, low variance), large depth gives many tiny boxes whose means track noise (low bias, exploding variance). The left panel of Figure 8 shows both ends — training MSE to 0, validation MSE above the constant baseline. `min_samples_leaf` attacks the same trade-off from the other end by bounding leaf population directly; since a leaf mean has variance σ^2/n_{leaf}, a floor of m caps that variance regardless of where the leaf sits in the tree. That makes it the better-targeted control of the two, and the experiments bear this out: the best depth-only model reaches validation MSE 2955.50, while adding a leaf floor reaches 2940.03. Depth is only a proxy for leaf population — a deep branch through a dense region can still hold well-populated leaves, while a shallow branch through a sparse one may not — and the leaf floor removes the proxy.

4 Conclusions

1. On Iris, entropy and Gini are interchangeable: identical trees up to depth 3, identical best validation accuracy (0.9545), with Gini preferred only on mean validation accuracy across the grid and on cost.

2. Overfitting begins beyond `max_depth` = 3: training accuracy climbs to 1.0000 while validation accuracy stops improving. Pre-pruning at depth 3 and post-pruning at $\alpha = 0.00899$ both recover the same validation accuracy, the latter with 4 leaves instead of 5.

3. On the 23-sample test set the two selected classifiers score 1.0000 and 0.9130 accuracy, a two-sample difference whose Wilson intervals overlap heavily; they are statistically indistinguishable and the report does not claim a winner.

4. On Diabetes the unrestricted tree is *worse than the constant predictor* on both validation and test, while the selected regularised tree (max_depth=10, min_samples_leaf=50) achieves test MSE 3668.44, RMSE 60.57 and MAE 48.11 — a 36% MSE reduction against the baseline. Every gain here came from constraining the tree.

5. `min_samples_leaf` proved the more effective control: the selected model's depth cap of 10 never binds, with the leaf floor stopping growth at depth 3 and 5 leaves.

6. Both mechanisms the sheet asks about were verified numerically rather than asserted: the root split reproduced by brute-force MSE reduction, and the leaf-mean rule confirmed to a maximum discrepancy of exactly 0 across all leaves.

A Design decisions that go beyond the sheet, and why

1. **Seven depths and seven leaf-size values instead of the required four/six.** The extra points at the regularised end were added because the `min_samples_leaf` curve was still falling at $m = 50$; without $m = 100$ one could not tell whether the minimum had been reached or the grid had simply run out.

2. **The full cost-complexity path is used rather than four hand-picked α values**, including the final α that collapses the tree to its root. The path breakpoints are the only α values at which the optimal subtree changes, so any other grid either duplicates them or misses one.

3. **Tie-breaking rules were fixed before inspecting results** (smaller depth, larger α, higher mean validation accuracy for the criterion). With 5 of 14 classifier configurations tied on validation accuracy, an unstated rule would have amounted to choosing the model after seeing the answer.

4. **Wilson confidence intervals on test accuracy.** Not requested, but a 23-point test set with a two-sample difference cannot support a claim about which pruning strategy is better, and the report says so.

5. **A constant-predictor baseline for the regression.** Also not requested; without it an MSE of 5706.61 is an uninterpretable number.

6. **The joint (d, m) grid for Question 2 is reported but not used for selection**, so the test estimate stays honest. Selecting from 49 configurations and then reporting the test score of the winner would bias the estimate downward.

7. **No feature scaling**, for the monotone-invariance reason given in Section 1.

B Files submitted

- `Lab5-IS24BM008.ipynb` — the notebook, executed top to bottom with all outputs and figures captured. 23 code cells, no unexecuted or failing cell.
- `Lab5-IS24BM008.pdf` — this report.